

HUGGING FACE & STANFORD UNIVERSITY · ARXIV 2025

SmolVLM: Redefining Small and Efficient Multimodal Models

A family of compact vision-language models — 256M, 500M, and 2.2B parameters — achieving competitive performance at sub-1GB RAM through balanced compute allocation, aggressive token compression, and extended context length.

256M · 500M · 2.2B params

Sub-1GB inference

Fully open-source

13 benchmarks

80B+

Idelfics-80B GPU RAM - impractical on edge

27.7 GB

RAM required by Molmo-A1B-7B (efficient baseline)

7B-70B

Typical SOTA VLM parameter range

Large VLMs Are Powerful but Impractical for Edge Deployment

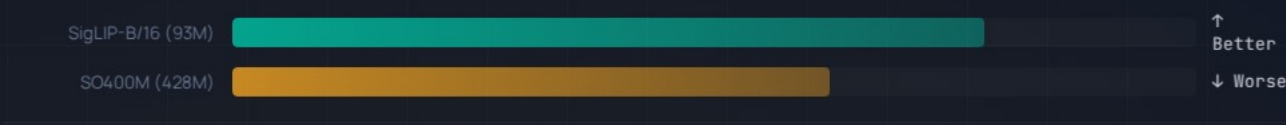
- › State-of-the-art VLMs demand **tens of GBs of GPU RAM**, making on-device use impossible
- › Even “efficient” models like Molmo-A1B-7B need **27.7 GB** – far beyond mobile hardware
- › Compression alone is insufficient; architectural decisions at design time determine deployability
- › SmolVLM targets **smartphones, edge devices, and privacy-preserving local inference** with no cloud dependency
- › Key question: *what design choices matter most when compute is extremely constrained?*

SmolVLM: Image Splitting → Pixel Shuffle → SmolLM2



Smaller Vision Encoders Outperform Larger Ones in Compact VLMs

AVERAGE BENCHMARK SCORE – 135M LM, VARYING ENCODER



PARAMETER OVERHEAD – ENCODER VS. LM



Key Result

At the smallest LM size (135M), the **93M SigLIP-B/16 encoder** achieves **higher benchmark scores** than the 428M SigLIP-SO400M – despite being 4.6× smaller.

Why it happens

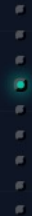
The 428M encoder’s features create a **training bottleneck** when paired with a tiny LM – the language model lacks capacity to exploit richer visual representations.

Scale boundary

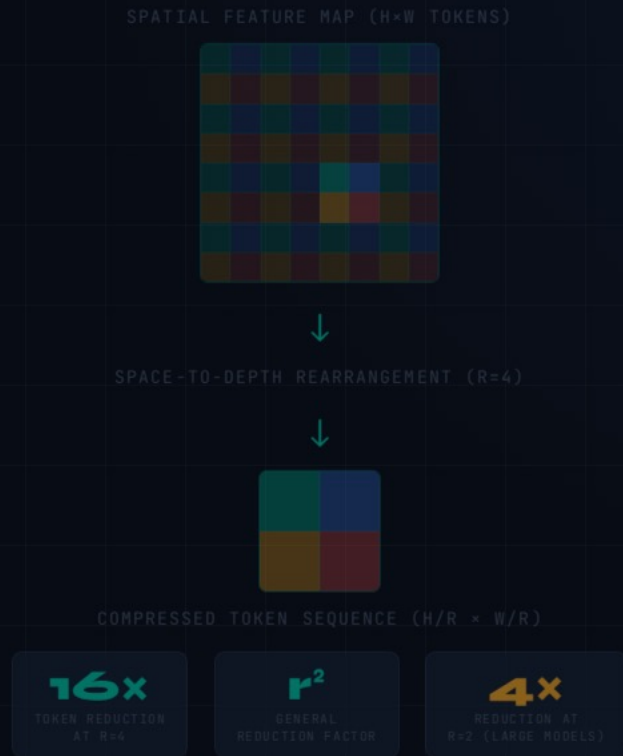
Only at **1.7B LM scale** does the larger encoder justify its 10% additional parameter overhead. Encoder choice must be matched to LM capacity.

Design principle

For compact VLMs: **balance encoder and LM parameter budgets**. Over-investing in the encoder when the LM is small wastes compute.

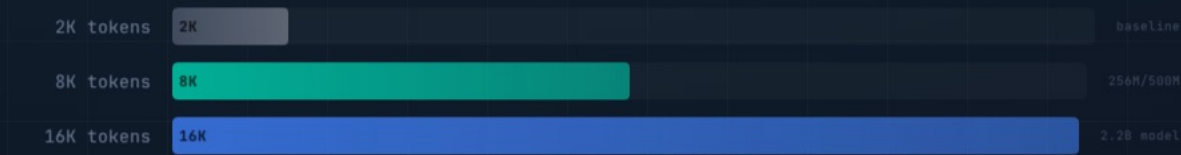


Aggressive Pixel Shuffle ($r=4$) Cuts Visual Tokens 16x for Small Models



Extending Context from 2K to 16K Tokens Unlocks Higher Image Resolution

CONTEXT WINDOW COMPARISON



Fine-tuned on mixed long/short-context data to maintain performance on both regimes.

Why context length matters for vision

Higher resolution images produce more visual tokens. Without sufficient context length, high-resolution inputs must be downsampled or truncated – degrading OCR, chart reading, and document understanding.

Technical strategy

RoPE base: 10K → 273K (27x increase) enables the positional encoding to generalize to sequences far beyond the original training length. Combined with mixed-context fine-tuning, this preserves performance on short sequences while unlocking long-context capability.

Model-specific context windows

SmoVLM-2.2B: 16K token context
SmoVLM-256M / 500M: 8K token context
Smaller models use 8K to balance memory and resolution needs.

Impact on benchmarks

Performance improves **significantly** when extending from 2K to the model's full context window – especially on document-heavy tasks (DocVQA, OCRBench) that benefit most from higher effective resolution.

SmolVLM: Competitive Performance at Sub-1GB to 4.9GB RAM

CATEGORY	BENCHMARK	256M	500M	2.2B	MOLMO-A1B-7B	INTERNL2-2B
SINGLE-IMAGE UNDERSTANDING	OCRBench	52.6%	61.0%	72.9%	54.7%	—
	AI2D	46.4%	59.2%	70.0%	71.0%	—
	ChartQA	55.6%	62.8%	68.7%	48.0%	—
	TextVQA	50.2%	60.2%	73.0%	61.5%	—
	DocVQA	58.3%	70.5%	80.0%	77.7%	—
	ScienceQA	73.8%	80.0%	89.6%	87.5%	—
MULTI-TASK REASONING	MMMU	29.0%	33.7%	42.0%	33.9%	—
	MathVista	35.9%	40.1%	51.5%	37.6%	—
	MMStar	34.6%	38.3%	46.0%	43.1%	—
VIDEO UNDERSTANDING	Video-MME	33.7%	42.2%	52.1%	—	45.0%
	MLVU	40.6%	47.3%	55.2%	—	48.2%
Average (11 tasks)		44.0%	51.0%	59.8%	—	—
GPU RAM (batch=1)		0.8 GB	1.2 GB	4.9 GB	27.7 GB	—

44.0%

256M avg
0.8 GB RAM

51.0%

500M avg
1.2 GB RAM

59.8%

2.2B avg
4.9 GB RAM

27.7 GB

Molmo-A1B-7B
RAM cost

300x

larger than LLaMA-80B
256M sur passes it

Real-Time Inference on Apple Silicon: Up to 80 tok/s

SMOLVLM-256M THROUGHPUT (TOKENS/SECOND, BATCH=1)



Throughput scales roughly linearly with batch size for the smallest model. Larger models (500M, 2.2B) show lower tok/s but remain usable on-device.

HuggingSnap iOS App

SmolVLM runs **fully on-device** via the HuggingSnap iOS app — processes photos and videos locally with **no cloud dependency**, ensuring privacy and low latency.

Why on-device matters

Privacy: no data leaves the device.

Latency: no network round-trip.

Cost: zero inference API cost.

Availability: works offline.

Model size vs. usability

At **~80 tok/s**, the 256M model produces a 200-token response in ~2.5 seconds — fast enough for interactive use. Even **55 tok/s on M3 Air** is responsive for most tasks.

Open-source infrastructure

All weights, datasets, training code, and the mobile app are **fully open-sourced** on Hugging Face to promote reproducibility and on-device AI research.

Design Principles for Efficient Multimodal Models

01

Balance Encoder-LM Compute

Match encoder size to LM capacity. **Smaller 93M encoder outperforms 428M SO400M** when paired with small LMs — don't over-invest in vision at the cost of language.

02

Use Aggressive Token Compression

For small VLMs, apply **pixel shuffle $r=4$ (16× reduction)**. Larger r is critical for fitting high-res inputs into memory; larger models may use $r=2$.

03

Extend Context Length

Push context from 2K to **8K–16K via RoPE base scaling** (10K → 273K) + mixed fine-tuning. Unlocks higher resolution and multi-image tasks without architecture changes.

04

Sub-1GB Inference Is Achievable

SmolVLM-256M uses only **0.8 GB RAM at batch=1** and surpasses the 300× larger Idefics-80B — architectural efficiency matters more than raw scale.

05

Open-Source End-to-End

Weights, training datasets, code, and a **production iOS app (HuggingSnap)** are all open-sourced — enabling reproducibility, research, and real deployments.